

LAB GUIDE WITH SOLUTIONS

Lab Guide

Collections

Ten labs with full solutions · approx. 6 hours

Elixir Training Programme

Elixir · collections

Session 8 · Instructor-led · Elixir 1.20.4 on Erlang/OTP 29

How to use this guide

Ten labs, in order, roughly six hours including discussion. Each lab states a goal, gives you the tasks, and then gives the full solution with the real output it produced on the training machine.

Session 2 measured what a tuple, a list and a map cost to HOLD. This session is what they cost to USE: create, read, add, update and remove; walk them with Enum, for and Stream; take them apart with patterns; and choose between them with numbers rather than habit. The data is a small bank ledger throughout — accounts, balances and twelve transactions — and lab 10 puts all of it together.

#	Lab	Minutes	Covers
1	Tuples and tagged results	30	elem, put_elem, {:ok, v} / {:error, why}
2	Lists	35	CRUD, out of range, prepend vs append, the charlist surprise
3	Keyword lists and options	30	Keyword.*, duplicates, validate!, the order trap
4	Maps	35	read six ways, put vs , update, merge, whole-map transforms
5	Structs and nested data	35	defstruct, @enforce_keys, get_in / update_in, Access
6	MapSet and ranges	30	set algebra, membership measured, ranges in guards
7	Iterating with Enum	40	group, reduce, rank, paginate, reduce_while, dates
8	Comprehensions and streams	35	for with filters and into:, Stream, File.stream!
9	Pattern matching on collections	35	nested heads, [h t] recursion, map keys, guards
10	Capstone: a bank ledger	50	fifteen events folded into structs, maps and a MapSet

IMPORTANT Windows users: the launchers are `iex.bat` and `mix.bat`. In PowerShell, plain `iex` collides with PowerShell's own Invoke-Expression alias — always type `iex.bat`.

WATCH OUT Four files sit in `code/`. `collection_lab.exs` carries labs 1 to 6; `enum_lab.exs` carries labs 7 to 9 and loads the twelve transactions from `transactions.exs`; `ledger.exs` is the capstone. All of them run as handed out — they print `:todo` where your code goes.

HOW OUTPUT IS SHOWN A list of small whole numbers prints as text: `[12, 11, 10]` shows as `~c"\f\v\n"`. Lab 2.4 explains why. So that a list of ids stays a list of ids, every output in this guide is printed with `charlists:as_lists`, and the starters' show helper does the same. In IEx, `IEx.configure(inspect: [charlists:as_lists])` does it for the whole session.

ERRORS ON PURPOSE Several tasks ask you to compile something that FAILS. Those are not mistakes in the guide — the error message is the lesson, and it is printed in full, in red. Read it before you read the explanation underneath.

LAB 1 · 30 MINUTES · TUPLES

Tuples and tagged results

GOAL Use a tuple for what it is good at — a fixed shape where position is the meaning — and see exactly where it stops being the right tool.

Task 1.1 — Read and "change" a tuple

An account as a tuple: `{"A-1", "Ada", 500}`. Read the balance, set it to 650 with `put_elem/3`, and print BOTH tuples afterwards.

Solution

```
acct = {"A-1", "Ada", 500}
elem(acct, 2)
richer = put_elem(acct, 2, 650)

elem(acct, 2)           = 500
richer = put_elem(acct, 2, 650) = {"A-1", "Ada", 650}
acct afterwards         = {"A-1", "Ada", 500}
tuple_size(acct)        = 3
```

WHY `put_elem` did not change `acct` — it returned a new three-slot tuple. Nothing in Elixir changes in place; the "update" is the return value, and if you do not bind it, it is gone. Deck 8 measured the cost: changing one slot copies the whole tuple.

Task 1.2 — The rest of the Tuple module — and the wall

```
Tuple.insert_at(acct, 3, :gold) = {"A-1", "Ada", 500, :gold}
Tuple.delete_at(acct, 1)       = {"A-1", 500}
Tuple.to_list(acct)            = ["A-1", "Ada", 500]

Enum.count(acct)
** (Protocol.UndefinedError) protocol Enumerable not implemented for Tuple
elem(acct, 3)
** (ArgumentError) errors were found at the given arguments:
* 1st argument: out of range
```

WHY A tuple is not Enumerable, on purpose: it is a record, not a sequence. If you want to walk it, you wanted a list. And "1st argument" names the index, not the tuple — `elem/2` is Erlang's `element/2`, which takes the index first.

WATCH OUT `Tuple.append/2` is deprecated in 1.20 — the compiler says "Use `insert_at` instead". The replacement for appending is `Tuple.insert_at(t, tuple_size(t), x)`.

Task 1.3 — Tagged results

Write `Parse.amount/1`. It turns text into a whole amount: `{:ok, n}` for good input, `{:error, reason}` for everything else — and the reason must say what was wrong. `Integer.parse/1` already returns a tagged result; case on it.

Solution

```
defmodule Parse do
  def amount(text) when is_binary(text) do
    case Integer.parse(text) do
```

```

    {n, ""} when n >= 0 -> {:ok, n}
    {_n, ""} -> {:error, :negative}
    {_n, rest} -> {:error, {:trailing, rest}}
    :error -> {:error, :not_a_number}
  end
end
end

Parse.amount("120") = {:ok, 120}
Parse.amount("12x") = {:error, {:trailing, "x"}}
Parse.amount("abc") = {:error, :not_a_number}
Parse.amount("-5") = {:error, :negative}
Parse.amount("0") = {:ok, 0}

results = Enum.map(["120", "abc", "75", "-5"], &Parse.amount/1)
results = [ok: 120, error: :not_a_number, ok: 75, error: :negative]
the good ones = for {:ok, n} <- results, do: n = [120, 75]
the bad ones = for {:error, why} <- results, do: why = [:not_a_number, :negative]

```

WHY Look at how results printed: [ok: 120, ...]. A list of two-element tuples whose first element is an atom IS a keyword list, so inspect shows it as one — the same data, the nicer syntax. And for with a pattern keeps only the elements that match; the next labs use that a lot.

Task 1.4 — Why records outgrow tuples

Add a fourth field — a tier — and try the pattern you wrote for three. Then do the same with a map.

```

{id, owner, balance} = {"A-1", "Ada", 500, :gold}
** (MatchError) no match of right hand side value:
    {"A-1", "Ada", 500, :gold}

m = %{id: "A-1", owner: "Ada", balance: 500}
%{balance: b} = Map.put(m, :tier, :gold)      # b = 500

```

WHY A tuple pattern fixes the size, so every pattern in the code base breaks the day a field is added. A map pattern names only the keys it needs. That is the whole case for maps and structs as records: a tuple for two or three values whose shape never changes, a map when the shape might.

CHECKPOINT You can say why `{:ok, value}` is idiomatic but `{"A-1", "Ada", 500, :gold, 3, [...]}` is not, and what `put_elem` would have to do to append one element.

LAB 2 · 35 MINUTES · LISTS

Lists

GOAL Change a list with every function you have, predict each result before you run it, and measure the one rule that matters most: the front is free and the back is not.

Task 2.1 — Create, add, update, remove

Start from `queue = [503, 501, 502]` — three transaction ids waiting to be processed. Predict each result, then run it.

Solution

```
[500 | queue]           = [500, 503, 501, 502]
queue ++ [504]          = [503, 501, 502, 504]
queue -- [501]          = [503, 502]
List.insert_at(queue, 1, 999) = [503, 999, 501, 502]
List.replace_at(queue, 0, 0)  = [0, 501, 502]
List.update_at(queue, 2, &(&1 + 1000)) = [503, 501, 1502]
List.delete_at(queue, -1)     = [503, 501]
Enum.sort(queue)             = [501, 502, 503]
queue (unchanged)           = [503, 501, 502]
```

WHY Negative indexes count from the end, as in `delete_at(queue, -1)`. And `queue` never changed: each line returned a new list. Only the first one — the prepend — reused the old list whole; every other line rebuilt at least part of it.

Task 2.2 — Out of range

Do the same operations at index 99. Which ones raise?

```
List.insert_at(queue, 99, :x) = [503, 501, 502, :x]
List.replace_at(queue, 99, :x) = [503, 501, 502]
List.delete_at(queue, 99)      = [503, 501, 502]
Enum.at(queue, 99)             = nil
Enum.at(queue, 99, :none)      = :none
List.first([])                 = nil

Enum.fetch!(queue, 99)
** (Enum.OutOfBoundsError) out of bounds error at position 99 when traversing
    enumerable [503, 501, 502]
hd([])
** (ArgumentError) errors were found at the given arguments:
    * 1st argument: not a nonempty list
```

WHY The index functions forgive a bad position: `insert_at` appends, `replace_at` and `delete_at` quietly do nothing, `Enum.at` returns `nil`. That is convenient and dangerous — a typo'd index is silently a no-op. When a missing element is a bug, use the `fetch!` version and let it crash.

Task 2.3 — Measure prepend against append

Write `Build.by_prepend/1` and `Build.by_append/1`, both returning `[1, 2, ..., n]`. One builds backwards with `[i | acc]` and reverses once; the other appends with `acc ++ [i]`. Check they agree, then time both at `n = 20,000`, each in a fresh process.

Solution

```
defmodule Build do
  def by_prepend(n), do: by_prepend(1, n, []) |> Enum.reverse()
  defp by_prepend(i, n, acc) when i > n, do: acc
  defp by_prepend(i, n, acc), do: by_prepend(i + 1, n, [i | acc])

  def by_append(n), do: by_append(1, n, [])
  defp by_append(i, n, acc) when i > n, do: acc
  defp by_append(i, n, acc), do: by_append(i + 1, n, acc ++ [i])
end

Build.by_prepend(5)      = [1, 2, 3, 4, 5]
Build.by_append(5)      = [1, 2, 3, 4, 5]
same result at 20,000?  = true
by_prepend(20_000)      614 us
by_append(20_000)       1748275 us
ratio                   2847x
```

WHY Same answer, 2,847 times the time. Each ++ copies every cell built so far, so building n elements copies about $n^2/2$ cells — here roughly 200 million. Deck 8 took it to 40,000 elements: 6.5 seconds against 0.8 milliseconds. Your numbers will differ; the ratio will not be small.

Task 2.4 — The charlist surprise

Run `Enum.map([72, 105], & &1)` in a script with plain `inspect`. Then print the good amounts from task 1.3 the same way.

```
inspect(Enum.map([72, 105], & &1))      = ~c"Hi"
inspect(for {:ok, n} <- results, do: n) = ~c"xK"
[72, 105] == ~c"Hi"                     = true

IO.inspect([72, 105], charlists: :as_lists)
[72, 105]
Enum.map([72, 105], &(&1 * 10))         = [720, 1050]
```

WHY Nothing is wrong with the data. A charlist — Erlang's string — IS a list of integers, so when every element is a printable character code, `inspect` assumes that is what you meant: 72 is H, 105 is i, 120 is x, 75 is K. The list is still `[72, 105]`; only the printing changed. `charlists: :as_lists` turns it off for one call, `IEx.configure(inspect: [charlists: :as_lists])` for a whole IEx session.

Task 2.5 — Test for empty without counting

Write `Q.empty?/1` and `Q.second/1` — `{:ok, x}` or `:error` — with patterns only. Then time `length(list) > 0` against your `empty?` on a 1,000,000-element list.

Solution

```
defmodule Q do
  def empty?([], do: true)
  def empty?([_ | _], do: false)

  def second([_, x | _], do: {:ok, x})
  def second(_, do: :error)
end

Q.empty?([])      = true
Q.empty?([1])     = false
```

```
Q.second([7, 8, 9]) = {:ok, 8}
Q.second([7])      = :error
length(1M list) > 0 2457 us
Q.empty?(1M list)   0 us
```

WHY `length/1` walks all million cells to answer a question about the first one. The pattern looks at one cell. `[_ , x | _]` says "at least two elements" and names the second — a list pattern fixes the front and does not care how long the rest is.

CHECKPOINT You can explain why `[x | list]` is free and `list ++ [x]` costs the whole list, and what `~c"x<"` means when `IEx` prints it.

LAB 3 · 30 MINUTES · KEYWORD LISTS

Keyword lists and options

GOAL Use a keyword list for what it is for — options to a function — and learn the three behaviours that surprise people: `put` moves the key, `keys` repeat, and patterns care about order.

Task 3.1 — The Keyword module

Solution

```
opts = [currency: "EUR", limit: 10]
opts[:currency]      = "EUR"
opts[:sort]          = nil
Keyword.get(opts, :sort, :desc) = :desc
Keyword.put(opts, :limit, 5)    = [limit: 5, currency: "EUR"]
Keyword.merge(opts, limit: 5)   = [currency: "EUR", limit: 5]
Keyword.delete(opts, :currency) = [limit: 10]

tags = [tag: "salary", tag: "monthly", limit: 3]
Keyword.get(tags, :tag)          = "salary"
Keyword.get_values(tags, :tag)   = ["salary", "monthly"]
Keyword.put(tags, :tag, "x")     = [tag: "x", limit: 3]
Keyword.keys(tags)              = [:tag, :tag, :limit]
```

WHY `Keyword.put/3` deletes every entry for the key and puts the new one at the FRONT — so `limit` moved, and the two tags collapsed to one. `Keyword.merge/2` kept the original order and replaced in place. `Keyword.get` returns the first match only; `get_values` returns them all. A map could hold none of this: its keys are unique and it keeps no order you can rely on.

Task 3.2 — A function that takes options

Write `Statement.render(amounts, opts \ \ [])`. It sorts the amounts, takes the first `:limit`, and formats each one with a currency. Options: `:currency` (default "EUR"), `:limit` (10) and `:sort` (`:desc`). An option you do not know must raise.

Solution

```
defmodule Statement do
  @defaults [currency: "EUR", limit: 10, sort: :desc]

  def render(amounts, opts \ \ []) do
    opts = Keyword.validate!(opts, @defaults)

    amounts
    |> Enum.sort(opts[:sort])
    |> Enum.take(opts[:limit])
    |> Enum.map(&("#{&1} #{opts[:currency]}")
  end
end

amounts = [500, 40, 120, 900, 60, 75]
Statement.render(amounts)
= ["900 EUR", "500 EUR", "120 EUR", "75 EUR", "60 EUR", "40 EUR"]
Statement.render(amounts, limit: 2)
= ["900 EUR", "500 EUR"]
```

```
Statement.render(amounts, sort: :asc, limit: 3, currency: "GBP")
= ["40 GBP", "60 GBP", "75 GBP"]

Statement.render(amounts, limit: 2)
** (ArgumentError) unknown keys [:limit] in [limit: 2], the allowed keys are:
   [:currency, :limit, :sort]
```

WHY `Keyword.validate!/2` does two jobs in one call: it fills in every default you did not pass, and it rejects every key it does not know. Without it, `limit: 2` would have been silently ignored and the caller would get ten lines and never know why. Note the call sites: trailing options need no square brackets — the parser wraps them into one last argument.

Task 3.3 — Do not pattern-match options

```
[currency: c, limit: l] = [limit: 5, currency: "EUR"]
** (MatchError) no match of right hand side value:
   [limit: 5, currency: "EUR"]
[currency: c] = [currency: "EUR", limit: 5]
** (MatchError) no match of right hand side value:
   [currency: "EUR", limit: 5]

o = [limit: 5, currency: "EUR"]
Keyword.fetch!(o, :currency) = "EUR"
Keyword.fetch!(o, :sort)
** (KeyError) key :sort not found in:
   [limit: 5, currency: "EUR"]
```

WHY A keyword list is a list, so a pattern for it is a list pattern: the same length, in the same order. Callers pass options in any order and any subset. Read options with `opts[:key]`, `Keyword.get/3` or `Keyword.fetch!/2` — never with `=`.

CHECKPOINT You can say why `Statement.render/2` takes options as a keyword list and not a map, and what `Keyword.validate!/2` protects you from.

LAB 4 · 35 MINUTES · MAPS

Maps: read, add, update, remove

GOAL Know which of the map's many access functions raises, which returns nil and which returns your default — and pick the one that says what you mean.

Task 4.1 — Read a key that is missing, six ways

Solution

```
acct = %{id: "A-1", owner: "Ada", balance: 500}
acct.balance           = 500
acct[:balance]         = 500
acct[:tier]            = nil
Map.get(acct, :tier, :standard) = :standard
Map.fetch(acct, :tier)   = :error

acct.tier
** (KeyError) key :tier not found in:
  %{id: "A-1", owner: "Ada", balance: 500}
Map.fetch!(acct, :tier)
** (KeyError) key :tier not found in:
  %{id: "A-1", owner: "Ada", balance: 500}
```

WHY Three answers for a missing key: raise (`., fetch!`), nil (`[]`), or something you chose (`Map.get/3`, `Map.fetch/2`). Use the dot for keys that must be there — the `KeyError` is the right crash, and it shows you the map. Use brackets or `Map.get/3` for keys that may not be.

Task 4.2 — Add, replace, count

Give the account a tier with `Map.put/3`, then with the `%{acct | ...}` syntax. Count visits with `Map.update/4`: the first call sets 1, later calls add one.

Solution

```
Map.put(acct, :tier, :gold) = %{id: "A-1", owner: "Ada", balance: 500, tier: :gold}
%{acct | balance: 650}     = %{id: "A-1", owner: "Ada", balance: 650}
%{acct | tier: :gold}
** (KeyError) key :tier not found

visit = fn a -> Map.update(a, :visits, 1, &(&1 + 1)) end
acct |> visit.() |> visit.() |> visit.()
= %{id: "A-1", owner: "Ada", balance: 500, visits: 3}
Map.update!(acct, :visits, &(&1 + 1))
** (KeyError) key :visits not found in:
  %{id: "A-1", owner: "Ada", balance: 500}
```

WHY `%{m | k: v}` is an update AND an assertion: it refuses to add a key. That makes it the right tool for records, where a new key is a typo. `Map.update/4` is the counter idiom — the default is stored as-is the first time, and the function runs only when the key already exists.

Task 4.3 — Remove and combine

January's and February's net movement per account are two maps. Merge them so an account in both months gets the sum.

Solution

```
Map.delete(acct, :owner)      = %{id: "A-1", balance: 500}
Map.drop(acct, [:owner, :balance]) = %{id: "A-1"}
Map.take(acct, [:id, :balance]) = %{id: "A-1", balance: 500}
{bal, rest} = Map.pop(acct, :balance)
= {500, %{id: "A-1", owner: "Ada"}}

jan = %{ "A-1" => 500, "B-2" => 40 }
feb = %{ "A-1" => -45, "C-3" => 900 }
Map.merge(jan, feb)
= %{ "A-1" => -45, "B-2" => 40, "C-3" => 900 }
Map.merge(jan, feb, fn _k, a, b -> a + b end)
= %{ "A-1" => 455, "B-2" => 40, "C-3" => 900 }
```

WHY Plain `Map.merge/2` lets the right-hand map win, so January's 500 was simply lost. The three-argument form calls your function only for keys in both maps — here, only for A-1.

Task 4.4 — Transform every entry, and stay a map

Apply 2% interest to every balance — integer arithmetic, `div(b * 102, 100)` — and then keep only balances over 100. Do each once with `Map` and once with `Enum`, and compare what comes back.

Solution

```
balances = %{ "A-1" => 455, "B-2" => 5, "C-3" => 750, "D-4" => 1150 }
Map.new(balances, fn {id, b} -> {id, div(b * 102, 100)} end)
= %{ "A-1" => 464, "B-2" => 5, "C-3" => 765, "D-4" => 1173 }
Map.filter(balances, fn {_id, b} -> b > 100 end)
= %{ "A-1" => 455, "C-3" => 750, "D-4" => 1150 }

Enum.filter(balances, fn {_id, b} -> b > 100 end)
= [ { "A-1", 455 }, { "C-3", 750 }, { "D-4", 1150 } ]
Enum.map(balances, fn {id, b} -> {id, div(b * 102, 100)} end)
= [ { "A-1", 464 }, { "B-2", 5 }, { "C-3", 765 }, { "D-4", 1173 } ]
Enum.sum(Map.values(balances)) = 2360
```

WHY `Enum` walks a map as `{key, value}` tuples and always hands back a list. `Map.new/2`, `Map.filter/2` and `Map.reject/2` keep it a map. (`Map.map/2` did the same job until 1.20 deprecated it: "Use `Map.new/2` instead".) B-2 stayed at 5 because `div(510, 100)` is 5 — integer money rounds down, which is a decision a real bank writes down.

Task 4.5 — Key order

```
%{zeta: 1, alpha: 2, mid: 3}      = %{zeta: 1, alpha: 2, mid: 3}
Map.keys(%{zeta: 1, alpha: 2, mid: 3}) = [:zeta, :alpha, :mid]
Map.keys(...) |> Enum.sort()        = [:alpha, :mid, :zeta]
%{ "b" => 1, "a" => 2 }              = %{ "a" => 2, "b" => 1 }
```

WHY String keys came back sorted; atom keys did not. A small map orders atom keys by when the atom was created — these three were created in the order you typed them — and a map past 32 keys uses hash order. None of it is a promise. If order matters, sort, or use a list.

CHECKPOINT For each of `m.k`, `m[:k]`, `%{m | k: v}` and `Map.update/4` you can say what happens when the key is missing.

LAB 5 · 35 MINUTES · STRUCTS · ACCESS

Structs and nested data

GOAL Turn the account into a struct so the compiler checks its fields, then update a value three levels deep without writing three levels of Map.put.

Task 5.1 — Define, build, and accept only the real thing

In `collection_lab.exs`, define `Account` with enforced `:id` and `:owner`, a balance defaulting to 0 and tags defaulting to `[]`. Write `Lab5.deposit/2` so that it accepts only an `%Account{}` and a positive amount.

Solution

```
defmodule Account do
  @enforce_keys [:id, :owner]
  defstruct [:id, :owner, balance: 0, tags: []]
end

defmodule Lab5 do
  def ada, do: %Account{id: "A-1", owner: "Ada", balance: 500}
  def deposit(%Account{} = a, amount) when amount > 0,
    do: %{a | balance: a.balance + amount}
end

Lab5.ada() = %Account{id: "A-1", owner: "Ada", balance: 500, tags: []}
Lab5.deposit(a, 100).balance = 600
Lab5.deposit(%{balance: 1}, 100)
** (FunctionClauseError) no function clause matching in Lab5.deposit/2
```

Now break it twice, on purpose — a missing enforced key, and a misspelt field:

```
def blank, do: %Account{id: "Z-9"}
** (ArgumentError) the following keys must also be given when building struct
    Account: [:owner]
    expanding struct: Account.__struct__/1

def typo, do: %Account{id: "Z-9", owner: "Zed", balance: 10}
** (KeyError) key :balance not found
    expanding struct: Account.__struct__/1
```

WHY "expanding struct" — both errors happen while the module that BUILDS the struct is compiling. A misspelt field never reaches a test, let alone production. A plain map would have accepted `balance: 10` without a word.

Task 5.2 — The script trap

Put `a = %Account{id: "A-1", owner: "Ada"}` at the top level of the same script that defines `Account` — outside any module — and run it.

```
defmodule Account do
  @enforce_keys [:id, :owner]
  defstruct [:id, :owner, balance: 0]
end

a = %Account{id: "A-1", owner: "Ada"}

error: cannot access struct Account, the struct was not yet defined or the struct
```

```
is being accessed in the same context that defines it
** (CompileError) ...: cannot compile file (errors have been logged)

a = struct!(Account, id: "A-1", owner: "Ada")      # the same script, fixed
%Account{id: "A-1", owner: "Ada", balance: 0}
```

WATCH OUT A struct literal is expanded when the code around it is COMPILED, and in 1.20 the top level of a .exs file is compiled in the same context as the defmodule above it — before that module exists. The error says exactly that. So in a script: build structs inside a function of another module (as Lab5.ada/0 does), or at the top level with struct!/2 — a function call, checked when it runs. A struct PATTERN at the top level fails the same way: the lab 10 harness hit it too. In a Mix project each module is compiled before its callers, and the problem does not arise.

Task 5.3 — What still works on a struct

```
struct(Account, id: "B", owner: "B", colour: :red)
= %Account{id: "B", owner: "B", balance: 0, tags: []}      # :colour dropped
struct!(Account, id: "B", owner: "B", colour: :red)
** (KeyError) key :colour not found

Map.put(a, :colour, :red)
= %{id: "A-1", owner: "Ada", balance: 500, __struct__: Account, tags: [], colour: :red}

a[:balance]
** (UndefinedFunctionError) function Account.fetch/2 is undefined (Account does
    not implement the Access behaviour)
```

WHY struct/2 silently drops unknown keys; struct!/2 refuses them — prefer the bang. Map.put/3 happily adds a field the struct never declared, and inspect no longer even prints it as an %Account{:}: it is a map with a __struct__ key now. And a struct deliberately refuses a[:balance] — use a.balance.

Task 5.4 — Three levels deep

Lab5.bank() returns a map holding a map of accounts. Add 60 to B-2's balance and give A-1 a second tag — each in ONE expression. Then do it with the account id in a variable.

Solution

```
bank = Lab5.bank()
bank.accounts["A-1"].balance      = 500
b2 = update_in(bank.accounts["B-2"].balance, &(&1 + 60))
b2.accounts["B-2"].balance      = 100
b3 = put_in(bank.accounts["A-1"].tags, [:vip, :gold])
b3.accounts["A-1"].tags          = [:vip, :gold]
bank.accounts["B-2"].balance     (unchanged) = 40

id = "A-1"
update_in(bank, [:accounts, id, Access.key!(:balance)], &(&1 - 100))
... .accounts[id].balance        = 400

get_in(bank, [:accounts, "B-2", :balance])
** (UndefinedFunctionError) function Account.fetch/2 is undefined (Account does
    not implement the Access behaviour)
update_in(bank.accounts["Z-9"].balance, &(&1 + 1))
** (BadMapError) expected a map, got:
    nil
```

WHY The path syntax `bank.accounts["B-2"].balance` mixes the two access styles: dots for struct fields, brackets for map keys. The list form needs `Access.key!/1` for a struct field, because a bare `:balance` asks the struct for `Access`, which it refuses. And only the maps on the path were rebuilt — `bank` itself never changed.

Task 5.5 — Every account at once

Charge a fee of 5 on every account in one `update_in` call.

Solution

```
update_in(bank, [:accounts, Access.all(), Access.key!(:balance)], &(&1 - 5))
** (RuntimeError) Access.all/0 expected a list, got: %{"A-1" => %Account{...}, ...}

charged = update_in(bank, [:accounts, Access.values(), Access.key!(:balance)],
                    &(&1 - 5))
balances afterwards      = %{"A-1" => 495, "B-2" => 35}

get_in(Map.values(bank.accounts), [Access.all(), Access.key!(:owner)])
= ["Ada", "Bob"]
```

WHY `Access.all/0` walks a LIST. For every value of a map, the accessor is `Access.values/0`. The error message says exactly which one you reached for and what it got instead.

CHECKPOINT You can build an `%Account{}` in a script without the compile error, and update a field two maps down in one expression.

LAB 6 · 30 MINUTES · MAPSET · RANGES

MapSet and ranges

GOAL Answer "who is in both?" and "have I seen this?" with a set, measure why, and use a range as a sequence that costs nothing to hold.

Task 6.1 — Set algebra on account ids

Three lists of ids, with duplicates, as they might arrive from three reports. Turn each into a MapSet and answer four questions.

Solution

```

depositors = MapSet.new(["A-1", "B-2", "C-3", "A-1", "B-2"])
withdrawers = MapSet.new(["A-1", "C-3", "D-4", "C-3"])
premium    = MapSet.new(["C-3", "D-4", "E-5"])

MapSet.size(depositors)           = 3
MapSet.intersection(depositors, withdrawers)
  both deposited and withdrew    = MapSet.new(["A-1", "C-3"])
MapSet.difference(premium, depositors)
  premium but never deposited    = MapSet.new(["D-4", "E-5"])
MapSet.difference(withdrawers, depositors)
  withdrew but never deposited    = MapSet.new(["D-4"])
MapSet.union(depositors, premium)
  anyone at all                  = MapSet.new(["A-1", "B-2", "C-3", "D-4", "E-5"])
MapSet.subset?(withdrawers, depositors) = false
"E-5" in premium                  = true

```

WHY Duplicates vanished on the way in — five ids became three. D-4 withdrew without ever depositing; in a real ledger that is the line worth investigating, and it took one function call to find.

Task 6.2 — Measure membership

Make 10,000 ids as a list and as a MapSet. Time 1,000 lookups that all miss — the worst case for the list.

Solution

```

ids      = Enum.map(1..10_000, &"ID-#{&1}")
id_set   = MapSet.new(ids)
probes   = Enum.map(1..1000, &"XX-#{&1}")
{t_list, _} = :timer.tc(fn -> Enum.count(probes, &(&1 in ids)) end)
{t_set, _}  = :timer.tc(fn -> Enum.count(probes, &MapSet.member?(id_set, &1)) end)

list      106496 us
MapSet    204 us
ratio     522x

```

WHY `in` on a list compares against every element until it finds one — and a miss means all 10,000. The set hashes once. The deck measured a single lookup at 1,435x; this run, with string ids, 522x. Either way the rule is the same: if the question is "have I seen this?", the answer is a MapSet.

Task 6.3 — Ranges

Solution

```
Enum.to_list(0..100//25)      = [0, 25, 50, 75, 100]
Enum.to_list(10..0//-5)       = [10, 5, 0]
Enum.to_list(10..0//1)        = []
Range.size(1..1_000_000//3)    = 333334
Enum.sum(1..100)               = 5050

defmodule Limit do
  def check(amount) when amount in 1..10_000, do: :ok
  def check(amount) when is_integer(amount), do: {:error, :out_of_range}
end

Limit.check(500)      = :ok
Limit.check(0)        = {:error, :out_of_range}
Limit.check(20_000)   = {:error, :out_of_range}

Enum.to_list(10..0)
warning: 10..0 has a default step of -1, please write 10..0//-1 instead
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

WHY `10..0//1` with a step of `+1` is EMPTY — it does not count down. Writing `10..0` without a step still counts down, but 1.20 warns, because that guess is exactly the bug. A range in a guard compiles to two comparisons: it never builds a list of ten thousand numbers.

CHECKPOINT You can name the collection for "unique ids", "is this id known?" and "the numbers 1 to a million", and say what each costs to hold.

LAB 7 · 40 MINUTES · ENUM

Iterating with Enum

GOAL Answer a manager's questions about twelve transactions with Enum alone — totals, balances, rankings, pages, the first overdraft — and sort dates correctly.

Open `enum_lab.exs`. It loads `transactions.exs`: twelve transactions on four accounts, January to March 2026, each a map with `:id`, `:date`, `:acct`, `:type` and `:amount`. It also defines `signed`, which turns a transaction into a signed amount.

```
signed = fn
  %{type: :deposit, amount: a} -> a
  %{type: :withdrawal, amount: a} -> -a
end
```

Task 7.1 — Totals and counts

Solution

```
deposits = Enum.filter(txns, &(&1.type == :deposit))
length(deposits) = 7
Enum.sum_by(deposits, & &1.amount) = 2890
txns |> Enum.filter(&(&1.type == :withdrawal)) |> Enum.sum_by(& &1.amount)
= 1030
Enum.frequencies_by(txns, & &1.type) = %{deposit: 7, withdrawal: 5}
Enum.count(txns, &(&1.amount >= 500)) = 4
```

Task 7.2 — Per account

Solution

```
Enum.group_by(txns, & &1.acct, & &1.amount)
= %{"A-1" => [500, 120, 75, 500], "B-2" => [40, 60, 25], "C-3" => [900, 300, 150],
  "D-4" => [1200, 50]}

balances =
  Enum.reduce(txns, %{}, fn t, acc ->
    Map.update(acc, t.acct, signed.(t), &(&1 + signed.(t)))
  end)
= %{"A-1" => -45, "B-2" => 5, "C-3" => 750, "D-4" => 1150}

for {id, b} <- balances, b < 0, do: {id, owners[id], b}
= [{"A-1", "Ada", -45}]
```

WHY `group_by`'s third argument picks what to keep from each element — here just the amount. The balances are a `reduce` with a map as the accumulator and `Map.update/4` as the counter idiom from lab 4. And a `for` with a filter answers "who is in the red?" in one line.

Task 7.3 — Rankings

Solution

```
txns |> Enum.sort_by(& &1.amount, :desc) |> Enum.take(3)
|> Enum.map(&{&1.id, &1.amount}) = [{8, 1200}, {4, 900}, {1, 500}]
```

```
{lo, hi} = Enum.min_max_by(txns, &&1.amount)
{lo.id, hi.id} = {9, 8}
txns |> Enum.filter(&&1.type == :withdrawal) |> Enum.max_by(&&1.amount)
|> Map.take([:id, :acct, :amount]) = %{id: 10, acct: "A-1", amount: 500}
```

WHY There are two transactions of 500 — ids 1 and 10 — and the top three kept id 1, the one that came first: `sort_by` is stable. `min_max_by` walks the list once for both answers.

Task 7.4 — Pages and a numbered statement

Solution

```
txns |> Enum.chunk_every(5) |> Enum.at(1) |> Enum.map(&&1.id)
= [6, 7, 8, 9, 10]
txns |> Enum.chunk_every(5) |> length() = 3

txns
|> Enum.filter(&&1.acct == "A-1")
|> Enum.with_index(1)
|> Enum.map(fn {t, i} -> "#{i}. #{t.date} #{t.type} #{t.amount}" end)
|> Enum.each(&IO.puts/1)

1. 2026-01-05 deposit 500
2. 2026-01-12 withdrawal 120
3. 2026-02-03 deposit 75
4. 2026-03-04 withdrawal 500
```

WHY `with_index/2` numbers from 1 when you ask it to. The last page has only two transactions — `chunk_every` keeps a short final chunk unless you pass `:discard`.

Task 7.5 — The first overdraft

Write `first_overdraft/1`: walk one account's transactions in order and stop at the first one that takes the balance below zero. Return `{:overdrawn, txn_id, balance}`, or the final balance if it never goes red.

Solution

```
first_overdraft = fn acct_id ->
  txns
  |> Enum.filter(&&1.acct == acct_id)
  |> Enum.reduce_while(0, fn t, bal ->
    new = bal + signed(t)
    if new < 0, do: {:halt, {:overdrawn, t.id, new}}, else: {:cont, new}
  end)
end

first_overdraft("A-1") = {:overdrawn, 10, -45}
first_overdraft("B-2") = {:overdrawn, 5, -20}
first_overdraft("C-3") = 750
first_overdraft("D-4") = 1150
```

WHY B-2 went red at transaction 5 and recovered to 5 by March — which is why 7.2's final balances did not flag it. `reduce_while` stops walking; `reduce` would have had to finish the list. `{:cont, acc}` continues, `{:halt, acc}` stops and makes its value the result.

Task 7.6 — Sorting dates

Take the dates of transactions 4, 5, 9 and 12, newest first, and sort them.

```
dates      = [~D[2026-03-18], ~D[2026-03-01], ~D[2026-02-02], ~D[2026-01-15]]
Enum.sort(dates) = [~D[2026-03-01], ~D[2026-02-02], ~D[2026-01-15], ~D[2026-03-18]]
Enum.sort(dates, Date)
              = [~D[2026-01-15], ~D[2026-02-02], ~D[2026-03-01], ~D[2026-03-18]]
Enum.max(dates, Date)
              = ~D[2026-03-18]

txns |> Enum.sort_by(& &1.date, { :desc, Date }) |> Enum.take(3) |> Enum.map(& &1.id)
= [12, 11, 10]
Enum.group_by(txns, & &1.date.month, & &1.id)
= %{1 => [1, 2, 3, 4], 2 => [5, 6, 7, 8], 3 => [9, 10, 11, 12]}
```

WHY The plain sort is wrong, silently: 1, 2, 15, 18 — it sorted by DAY. A Date is a struct, and structs compare as maps, field by field. Passing Date tells Enum to use Date.compare/2. With sort_by, the sorter goes in a tuple with the direction.

CHECKPOINT You reached for filter, group_by, reduce, sort_by, chunk_every and reduce_while — and wrote no recursion and no loop.

LAB 8 · 35 MINUTES · FOR · STREAM

Comprehensions and streams

GOAL Filter, transform and collect in one for; then process data that is too big — or endless — to hold as a list.

Task 8.1 — for with filters, and patterns that skip

Five raw CSV lines, three of them bad. Parse each to `{:ok, {acct, type, n}}` or `{:error, line}`, then use one for to keep the good ones and another to keep the bad ones.

Solution

```
for %{type: :withdrawal, amount: a, id: id} <- txns, a >= 100, do: {id, a}
= [{3, 120}, {7, 300}, {10, 500}]

raw = ["A-1,deposit,100", "B-2,withdrawal,oops", "C-3,deposit,50",
      "garbage", "D-4,deposit,-5"]
parse_line = fn l ->
  with [acct, type, amt] <- String.split(l, ","),
       {n, ""} when n > 0 <- Integer.parse(amt) do
    {:ok, {acct, String.to_existing_atom(type), n}}
  else
    _ -> {:error, l}
  end
end
parsed = Enum.map(raw, parse_line)

for {:ok, t} <- parsed, do: t
= [{"A-1", :deposit, 100}, {"C-3", :deposit, 50}]
for {:error, l} <- parsed, do: l
= ["B-2,withdrawal,oops", "garbage", "D-4,deposit,-5"]
for a <- ["A-1", "B-2"], m <- [1, 2], do: {a, m}
= [{"A-1", 1}, {"A-1", 2}, {"B-2", 1}, {"B-2", 2}]
```

WHY A pattern on the left of `<-` is a filter: an element that does not match is skipped, not an error. `Enum.map` with a pattern that fails would raise instead. Two generators give every combination. `String.to_existing_atom/1` is safe here because `:deposit` and `:withdrawal` already exist — never turn raw input into NEW atoms (deck 5).

Task 8.2 — Collect into a shape

Solution

```
for {id, name} <- owners, into: %{}, do: {name, id}
= %{"Ada" => "A-1", "Bob" => "B-2", "Cy" => "C-3", "Dee" => "D-4"}
for %{type: :withdrawal, acct: a} <- txns, into: MapSet.new(), do: a
= MapSet.new(["A-1", "B-2", "C-3", "D-4"])
for t <- txns, uniq: true, do: t.date.month = [1, 2, 3]
for t <- Enum.take(txns, 3), into: "", do: "#{t.id}" = "[1][2][3]"

for t <- txns, reduce: %{} do
  acc -> Map.update(acc, t.acct, signed.(t), &(&1 + signed.(t)))
end
= %{"A-1" => -45, "B-2" => 5, "C-3" => 750, "D-4" => 1150}
```

WHY `into`: pours the results into any Collectable — a map, a set, a string. `uniq: true` drops repeats. `reduce`: turns the comprehension into a fold, and it produced exactly the balances `Enum.reduce` did in 7.2.

Task 8.3 — Lazy: stop when you have enough

1,000 at 7% a year, integer arithmetic: in which year does it first reach 2,000? Use `Stream.iterate/2` — you do not know in advance how many years to generate. Then find the first three "suspicious" amounts in two million, once with `Enum` and once with `Stream`.

Solution

```
Stream.iterate(1000, &div(&1 * 107, 100))
|> Stream.with_index()
|> Enum.find(fn {b, _year} -> b >= 2000 end)      = {2098, 11}
Stream.iterate(1000, &div(&1 * 107, 100)) |> Enum.take(4)
= [1000, 1070, 1144, 1224]

suspicious = fn n -> rem(n * 7919, 1000) == 0 end
1..2_000_000 |> Enum.map(&(&1 * 3)) |> Enum.filter(suspicious) |> Enum.take(3)
1..2_000_000 |> Stream.map(&(&1 * 3)) |> Stream.filter(suspicious) |> Enum.take(3)

Enum result      = [3000, 6000, 9000]
Stream result    = [3000, 6000, 9000]
Enum 139673 us
Stream 102 us
```

WHY `Stream.iterate` is infinite, and that is fine: `Enum.find` stops asking at year 11. The `Enum` pipeline built a two-million-element list, filtered all of it, and then kept three. The `Stream` pipeline looked at 3,000 numbers and stopped. Same answer, over a thousand times faster — because it did a thousandth of the work.

Task 8.4 — A file too big to read

Write 100,000 CSV lines to a temp file. Then compute the net total with `File.stream!/1`, never holding the file in memory.

Solution

```
csv = Path.join(System.tmp_dir!(), "lab8_ledger.csv")
File.write!(csv, ["acct,type,amount\n",
  for i <- 1..100_000, do: "A-#{rem(i, 50)},#{if rem(i, 4) == 0, do: "withdrawal",
    else: "deposit"},#{rem(i, 97) + 1}\n"}])
File.stat!(csv).size      = 1645739

csv
|> File.stream!()
|> Stream.drop(1)
|> Stream.map(&String.trim_trailing/1)
|> Stream.map(&String.split(&1, ","))
|> Enum.reduce(0, fn
  [_a, "deposit", n], acc -> acc + String.to_integer(n)
  [_a, "withdrawal", n], acc -> acc - String.to_integer(n)
end)

net of 100,000 lines      = 2449723
streamed in 157 ms
```

WHY `File.stream!/1` is a stream of lines. `Stream.drop(1)` skips the header. Each line goes through both maps and into the reduce before the next one is read, so memory holds one line, not 1.6 MB — and the same code would handle 16 GB. The reduce's fn has two clauses: a pattern per line shape, exactly as in lab 9.

CHECKPOINT You can say when a Stream is worth it — big, endless, or stopping early — and when plain Enum is simpler and faster (deck 8 measured a 100-element pipeline at 2.5x slower as a stream).

LAB 9 · 35 MINUTES · PATTERNS

Pattern matching on collections

GOAL Let the function head take a collection apart: one clause per shape of data, recursion over `[h | t]`, map keys you choose at run time, and guards that test collections.

Task 9.1 — One clause per response shape

An HTTP client returns `{:ok, %{ "status" => s, ... }}` or `{:error, reason}`. Write `Resp.handle/1` so each shape gets its own clause.

Solution

```
defmodule Resp do
  def handle({:ok, %{ "status" => 200, "data" => [] }}, do: :no_data)
  def handle({:ok, %{ "status" => 200, "data" => [%{ "id" => id } | _] = rows }},
    do: {:rows, length(rows), id})
  def handle({:ok, %{ "status" => s }} when s in 400..499, do: {:client, s})
  def handle({:ok, %{ "status" => s }} when s >= 500, do: {:retry, s})
  def handle({:error, reason}), do: {:failed, reason}
end

{:ok, %{ "status" => 200, "data" => [] }} = :no_data
{:ok, %{ "status" => 200, "data" => [%{ "id" => 7 }, %{ "id" => 8 }] }}
  = {:rows, 2, 7}
{:ok, %{ "status" => 404 }} = {:client, 404}
{:ok, %{ "status" => 503 }} = {:retry, 503}
{:error, :timeout} = {:failed, :timeout}

Resp.handle({:ok, %{ "status" => 302 }})
** (FunctionClauseError) no function clause matching in Resp.handle/1
```

WHY `[%{ "id" => id } | _] = rows` both checks the shape and names the whole list, so the clause can count it. The 302 crash is correct: nobody decided what a redirect means yet, and a clause that quietly returned something would hide that.

Task 9.2 — Recursion over a list, without Enum

Write `sum/1`, `count_by_type/1`, `largest/1` and `pairs/1` with `[h | t]` patterns only.

Solution

```
defmodule Mine do
  def sum([], do: 0)
  def sum([h | t]), do: h + sum(t)

  def count_by_type(list), do: count_by_type(list, %{})
  defp count_by_type([], acc), do: acc
  defp count_by_type([%{type: type} | rest], acc),
    do: count_by_type(rest, Map.update(acc, type, 1, &(&1 + 1)))

  def largest([h | t]), do: largest(t, h)
  defp largest([], best), do: best
  defp largest([%{amount: a} = h | t], %{amount: b}) when a > b, do: largest(t, h)
  defp largest([_ | t], best), do: largest(t, best)
```

```
def pairs([a, b | rest]), do: [{a, b} | pairs(rest)]
def pairs(_), do: []
end

Mine.sum([500, 40, 120])      = 660
Mine.count_by_type(txns)      = %{deposit: 7, withdrawal: 5}
Mine.largest(txns).id         = 8
Mine.pairs([1, 2, 3, 4, 5])    = [{1, 2}, {3, 4}]
Mine.largest([])
** (FunctionClauseError) no function clause matching in Mine.largest/1
```

WHY `pairs/1` looks two elements ahead; the leftover 5 falls to the catch-all. `largest/1` has no clause for `[]`, deliberately — the largest of nothing has no answer, and the crash says so. In real code these are `Enum.sum`, `Enum.frequencies_by`, `Enum.max_by` and `Enum.chunk_every`; writing them once shows you what those do.

Task 9.3 — Map patterns: partial, pinned, and the empty-map trap

Solution

```
%{owner: who, balance: bal} = acct      # {"Ada", 500}
field = :balance
%{^field => v} = acct                    # v = 500

defmodule Empty do
  def wrong(%{}), do: :empty
  def right(m) when map_size(m) == 0, do: :empty
  def right(m) when is_map(m), do: {:has, map_size(m)}
end

Empty.wrong(%{a: 1}) = :empty
Empty.right(%{}) = :empty
Empty.right(%{a: 1}) = {:has, 1}

field = :balance
%{field => v} = %{balance: 500}
error: cannot use variable field as map key inside a pattern. Map keys in
patterns can only be literals (such as atoms, strings, tuples, and the like) or
an existing variable matched with the pin operator (such as ^some_var)
```

WHY A map pattern names the keys it needs and ignores the rest — so `%{}` names none and matches EVERY map. To test for empty, guard on `map_size(m) == 0`. A key in a pattern must be known: a literal, or a variable you pin. A pattern cannot bind a key, only a value.

Task 9.4 — Struct patterns check the name

Solution

```
defmodule Only do
  def owner(%Account{owner: o}), do: o
end

Only.owner(Lab5.ada()) = "Ada"
Only.owner(%{owner: "Ada"})
** (FunctionClauseError) no function clause matching in Only.owner/1
```

WHY The plain map has the same key and the same value, and still does not match: `%Account{}` checks `__struct__` too. That is what makes a struct a type you can dispatch on.

Task 9.5 — Guards for collections

Solution

```
defmodule Tier do
  def label(acct) when is_map_key(acct, :tier), do: "tier #{acct.tier}"
  def label(%{balance: b}) when b >= 1000, do: "large"
  def label(%{}), do: "standard"
end

Tier.label(%{tier: :gold})      = "tier gold"
Tier.label(%{balance: 1150})    = "large"
Tier.label(%{balance: 5})       = "standard"
```

WHY The guard-safe collection tests: `is_list`, `is_map`, `is_tuple`, `map_size`, `tuple_size`, and `in` with a literal list or a range. `length/1` is allowed in a guard too — but it walks the list every time the clause is tried.

CHECKPOINT You can write a function whose clauses cover every shape of its input, with no `if` and no `case` in the body.

LAB 10 · 50 MINUTES · CAPSTONE

Capstone: a bank ledger

GOAL Fold fifteen raw events into a ledger — structs in a map, a list of rejections, a MapSet of flagged accounts — choosing each collection on purpose, and prove the books balance.

Open code/ledger.exs. Account and a Ledger struct are given, as are the fifteen events and a harness at the bottom that prints what the ledger can do. Every Ledger function is a stub that returns something harmless, so the file runs as handed out. Replace the stubs, top to bottom, and re-run after each one.

```
defmodule Account do
  @enforce_keys [:id, :owner]
  defstruct [:id, :owner, balance: 0, history: []]
end

defmodule Ledger do
  defstruct accounts: %{}, rejected: [], flagged: MapSet.new()

  def events do
    [
      {:open, "A-1", "Ada"},
      {:open, "B-2", "Bob"},
      {:deposit, "A-1", 500},
      {:deposit, "B-2", 40},
      {:withdraw, "A-1", 120},
      {:withdraw, "B-2", 60},
      {:deposit, "C-3", 900},
      {:open, "C-3", "Cy"},
      {:deposit, "C-3", 900},
      {:transfer, "C-3", "A-1", 300},
      {:open, "A-1", "Imposter"},
      {:bogus, 1},
      {:withdraw, "A-1", 75},
      {:deposit, "B-2", -5},
      {:transfer, "A-1", "B-2", 10_000}
    ]
  end
end
```

Piece of state	Collection	Why this one
one account	struct %Account{}	fixed, known fields; typos fail at compile time
all accounts	map, id => %Account{}	lookup by id on every event, at any size
an account's history	list, newest first	prepend is free; reverse once to print
an event	tuple {tag, id, n}	a fixed shape per kind; the tag picks the clause
rejections	list of {event, reason}	in arrival order: prepend, reverse at the end
flagged accounts	MapSet	each id once, however many times it failed
report options	keyword list	optional, any order, validated

Task 10.1 — One clause per event

Write `apply_event(ledger, event)`. It returns `{:ok, new_ledger}` or `{:error, reason}`, with one clause per event shape. The reasons the harness expects are `:already_open`, `:no_account`, `id`, `:insufficient_funds`, `:bad_amount` and `:malformed`. Record each movement by PREPENDING `{what, signed_amount}` to the account's history.

Solution

```
def apply_event(%Ledger{} = l, {tag, id, owner}) when is_binary(id) and is_binary(owner) do
  if Map.has_key?(l.accounts, id),
    do: {:error, :already_open},
    else: {:ok, put_account(l, %Account{id: id, owner: owner})}
end

def apply_event(%Ledger{} = l, {tag, id, amount})
```

```

    when is_integer(amount) and amount > 0 do
      with {:ok, a} <- fetch(l, id) do
        {:ok, put_account(l, move(a, amount, :deposit))}
      end
    end
  end

  def apply_event(%Ledger{} = l, {:withdraw, id, amount})
    when is_integer(amount) and amount > 0 do
    with {:ok, a} <- fetch(l, id),
         :ok <- covers(a, amount) do
      {:ok, put_account(l, move(a, -amount, :withdraw))}
    end
  end

  def apply_event(%Ledger{} = l, {:transfer, from, to, amount})
    when is_integer(amount) and amount > 0 and from != to do
    with {:ok, f} <- fetch(l, from),
         {:ok, t} <- fetch(l, to),
         :ok <- covers(f, amount) do
      {:ok, l |> put_account(move(f, -amount, {:to, to}))
        |> put_account(move(t, amount, {:from, from}))}
    end
  end

  def apply_event(%Ledger{}, {kind, _id, amount})
    when kind in [:deposit, :withdraw] and is_integer(amount),
    do: {:error, :bad_amount}
  def apply_event(%Ledger{}, _event), do: {:error, :malformed}

  defp fetch(%Ledger{accounts: accounts}, id) do
    case Map.fetch(accounts, id) do
      {:ok, a} -> {:ok, a}
      :error -> {:error, {:no_account, id}}
    end
  end

  defp covers(%Account{balance: b}, amount) when b >= amount, do: :ok
  defp covers(%Account{}, _amount), do: {:error, :insufficient_funds}

  defp move(%Account{} = a, amount, what),
    do: %{a | balance: a.balance + amount, history: [{what, amount} | a.history]}

  defp put_account(%Ledger{} = l, %Account{id: id} = a), do: put_in(l.accounts[id], a)

```

```

--- 10.1 single events
open A-1, then its balance      0
deposit 500 into A-1            500
withdraw 50 from empty A-1     {:error, :insufficient_funds}
deposit into Z-9                {:error, {:no_account, "Z-9"}}
open A-1 again                  {:error, :already_open}
deposit -5                      {:error, :bad_amount}
{:bogus, 1}                    {:error, :malformed}
transfer to self                {:error, :malformed}

```

WHY The event's tag picks the clause and its guard checks the amount; with chains the lookups and the funds check, and any `{:error, _}` falls straight out as the result. Clause ORDER matters twice: a negative deposit must fail the first deposit clause before it can reach `:bad_amount`, and a transfer to yourself fails the `from != to` guard and lands on `:malformed`.

WATCH OUT `%Ledger{}` helper is fine inside the module. At the top level of the script — the harness — a `%Ledger{}` pattern is the same compile error as lab 5.2, which is why the harness matches on `{:ok, _}` and trusts the shape.

Task 10.2 — Fold all fifteen events

Write `apply_all/1`. Reduce over the events; on `{:ok, _}` keep the new ledger, on `{:error, why}` prepend `{event, why}` to `:rejected` and add every id the event names to `:flagged`. Reverse `:rejected` once at the end.

Solution

```
def apply_all(events) do
  l =
    Enum.reduce(events, new(), fn event, l ->
      case apply_event(l, event) do
        {:ok, l} -> l
        {:error, why} ->
          %{l | rejected: [{event, why} | l.rejected],
            flagged: MapSet.union(l.flagged, MapSet.new(ids_in(event)))}
      end
    end)

  %{l | rejected: Enum.reverse(l.rejected)}
end

defp ids_in({:transfer, from, to, _}), do: [from, to]
defp ids_in({_kind, id, _}) when is_binary(id), do: [id]
defp ids_in(_), do: []
```

```
--- 10.2 all fifteen events
accounts  ["A-1", "B-2", "C-3"]
A-1       %Account{id: "A-1", owner: "Ada", balance: 605, history: [{:withdraw, -75},
                        {:from, "C-3"}, 300}, {:withdraw, -120}, {:deposit, 500}]}
rejected:
{:withdraw, "B-2", 60}           :insufficient_funds
{:deposit, "C-3", 900}          :no_account, "C-3"
{:open, "A-1", "Imposter"}      :already_open
{:bogus, 1}                     :malformed
{:deposit, "B-2", -5}           :bad_amount
{:transfer, "A-1", "B-2", 10000} :insufficient_funds
flagged  MapSet.new(["A-1", "B-2", "C-3"])
```

WHY Six rejections, in the order they arrived, because they were prepended and reversed once. B-2 failed three times and A-1 twice, and each appears in `flagged` once — that is the MapSet doing its job. C-3's first deposit arrived before the account existed; the second one, after `:open`, went through.

Task 10.3 — Reports — and prove the books balance

Write `balances/1` (richest first), `total/1` and `statement/2` (oldest first). Then check the ledger: money deposited minus money withdrawn, over the ACCEPTED events only, must equal the total.

Solution

```
def balances(%Ledger{accounts: a}) do
  a |> Map.values() |> Enum.sort_by(& &1.balance, :desc)
  |> Enum.map(&{&1.id, &1.owner, &1.balance})
end

def total(%Ledger{accounts: a}), do: a |> Map.values() |> Enum.sum_by(& &1.balance)

def statement(%Ledger{} = l, id) do
  l.accounts[id].history
  |> Enum.reverse()
  |> Enum.with_index(1)
  |> Enum.map(fn {what, amount}, i -> "#{i}. #{label(what)} #{amount}" end)
end

accepted = Ledger.events() -- Enum.map(l.rejected, &elem(&1, 0))
money_in = for {:deposit, _, n} <- accepted, do: n
money_out = for {:withdraw, _, n} <- accepted, do: n
```

```
--- 10.3 reports
balances      [{"A-1", "Ada", 605}, {"C-3", "Cy", 600}, {"B-2", "Bob", 40}]
total         1245
accepted events 9
in - out      1245
statement for A-1:
  1. deposit 500
  2. withdrawal -120
  3. transfer from C-3 300
  4. withdrawal -75
```

WHY The books balance: 1,245 in the accounts, 1,245 in minus out. Transfers move money between accounts and add nothing, which is why the check ignores them.

WATCH OUT The first draft of this check filtered the events with `Enum.any?(l.rejected, fn {r, _} -> r == e end)` — and got 345, not 1,245. `{:deposit, "C-3", 900}` arrives twice, rejected once and accepted once, and the two tuples are EQUAL, so the filter threw both away. `--` removes the FIRST match only, once per element on its right — exactly the semantics this needs. Deck 8 flagged `--`'s first-match rule as a surprise; here it is the fix.

Task 10.4 — A report that takes options

Write `report(ledger, opts \ \ [])` with options `sort: :balance` (default, richest first) or `:id`, and `min_balance: (default 0)`. An unknown option must raise.

Solution

```
def report(%Ledger{} = l, opts \ \ []) do
  opts = Keyword.validate!(opts, sort: :balance, min_balance: 0)

  rows =
    for %Account{id: id, owner: o, balance: b} <- Map.values(l.accounts),
      b >= opts[:min_balance],
      do: {id, o, b}

  case opts[:sort] do
    :balance -> Enum.sort_by(rows, &elem(&1, 2), :desc)
    :id -> Enum.sort_by(rows, &elem(&1, 0))
  end
end
```

```
end
```

```
--- 10.4 report with options
report(l)                [{"A-1", "Ada", 605}, {"C-3", "Cy", 600}, {"B-2", "Bob", 40}]
report(l, sort: :id)      [{"A-1", "Ada", 605}, {"B-2", "Bob", 40}, {"C-3", "Cy", 600}]
report(l, min_balance: 100) [{"A-1", "Ada", 605}, {"C-3", "Cy", 600}]
report(l, sort_by: :id)
** (ArgumentError) unknown keys [:sort_by] in [sort_by: :id], the allowed keys are:
    [:sort, :min_balance]
```

WHY The `for` here uses a struct pattern in the generator, a filter, and a tuple per row — a fixed shape. A case with no clause for an unknown `:sort` value would crash, which is right: `validate!` has already rejected the key, and a bad value is a bug in the caller.

CHECKPOINT Every collection in your ledger is there for a reason you can state — the table at the start of this lab — and the books balance.

Where this goes next

You can now pick a collection by what it costs, change it without mutating it, walk it with the right tool, and take it apart in a function head. Session 9 is macros — `quote`, `unquote` and writing your own `__using__` — code that runs at compile time and returns code, which is how `defstruct`, `if` and the pipe were built in the first place.

If you remember one thing per lab	It is this
1 · Tuples	a fixed shape; <code>{:ok, v}</code> / <code>{:error, why}</code> ; not Enumerable
2 · Lists	<code>prepend</code> is free, <code>append</code> copies; build backwards, reverse once
3 · Keyword lists	options: ordered, repeatable, validated — never pattern-matched
4 · Maps	dot for must-have keys, <code>[]</code> for maybe; <code> </code> updates only existing keys
5 · Structs	keys checked at compile time; path syntax for nested updates
6 · MapSet and ranges	membership in constant time; a range costs 12 words
7 · Enum	<code>group_by</code> , <code>reduce</code> , <code>sort_by</code> , <code>reduce_while</code> — and sort dates with <code>Date</code>
8 · for and Stream	a pattern in a generator filters; streams stop early
9 · Patterns	one clause per shape; <code>%{}</code> matches every map
10 · Capstone	choose each collection on purpose, then prove the result

The one-line rule

Pick the collection by the operation you will do most — walk it, look it up, test membership, or hold a fixed shape — and remember that every change returns a new one.

MEASURED Every output in this guide was produced by `../verify/probe_labs8.exs` and `../verify/ledger_solution.exs` on Elixir 1.20.4 / Erlang/OTP 29. Every compiler error and warning came from a real compile in a child VM. Timings are one run on the training machine: yours will differ, and the ratios are the lesson. If your outputs disagree, your machine is right — say so and work out why.